

TALON: HUMAN DETECTION & REAL-TIME TRACKING PIPELINE

Technical Specification & System Documentation

Aerial Telemetry, RF-DETR Transformer Detection, & DeepSORT Target Lock

End-to-End Data Processing, Model Training, Evaluation, and Real-Time Operator
Dashboard

Author: Aadarsh Sinha and Krishnashish Das

Repository: [e:\Human Detection Dataset](#)

Document Revision: 1.0

Technical Report

Document Control

Field	Value
Document Title	Talon: Human Detection Dataset & Real-Time Tracking Pipeline
Document ID	DOC-HDD-2026-v1.0
Version	1.0
Status	Released
Author	Aadarsh Sinha and Krishnashish Das
Organization	Aerial Systems & Autonomous Robotics Lab
Release Date	August 26, 2026
Last Updated	August 26, 2026
Distribution	Engineering & Field Ops Teams
Classification	Technical Specification

Note

System Overview: This technical document specifies the data conversion, dataset partitioning, deep learning architecture (RF-DETR / RT-DETR), and DeepSORT target locking engine for aerial human detection under low-bandwidth 480p drone feeds.

Revision History

Version	Date	Author(s)	Changes
1.0	August 26, 2026	Aadarsh Sinha, Krishnashish Das	Initial release of dataset conversion, RF-DETR fine-tuning, and interactive target lock demo.

Contents

Document Control	1
Revision History	2
Abstract	4
1 Introduction	5
1.1 Overview	5
1.2 Design Goals	5
2 Architecture Overview	6
2.1 System Dataflow Architecture	6
2.2 Directory Component Summary	6
3 Specification & Data Models	7
3.1 DeepSORT Tracker Parameters	7
4 Implementation Code Highlights	8
4.1 VoTT to YOLO Bounding Box Conversion	8
4.2 Interactive Target Locking Mouse Callback	8
5 Performance & Benchmarks	9
5.1 Model Metrics Summary	9
5.2 Live Real-Time Webcam Demo	9
5.3 Confusion Matrix Benchmark Script	9
6 Operational & Safety Considerations	11
6.1 Low-Bandwidth Transmission Optimization	11
6.2 Operator Display Upscaling	11
A Appendix A: Glossary	12
B Appendix B: Python Dependencies	12

Abstract

Real-time human detection and tracking from aerial drone platforms present unique computer vision challenges, including low pixel resolution, extreme perspective distortion, scale variation, and motion blur caused by drone vibrations and gimbal movement.

This documentation presents the complete engineering specification for the **Human Detection & Tracking Pipeline**. The system processes the NTUT Drone Dataset, converts heterogeneous VoTT CSV annotations into normalized YOLO and RF-DETR formats, trains modern transformer-based object detectors (**RF-DETR** and **RT-DETR**), evaluates model performance via confusion matrices, and provides an interactive real-time Ground Control Station (GCS) tracking demo with **DeepSORT** feature embeddings and visual target locking.

Key system capabilities include:

- **Automated VoTT-to-YOLO Converter:** Converts multi-class VoTT CSV annotations (`walk`, `stand`, `push`, `watchphone`, `block`) into normalized single-class human detection labels (`class_id = 0`).
- **RF-DETR & RT-DETR Training Modules:** Transformer-based detection engines fine-tuned specifically for small-object aerial imagery at 640×480 input resolution.
- **DeepSORT Target Locking & Visual ReID:** Integrates MobileNet visual embeddings with Kalman filter tracking to allow operators to lock onto individual targets via mouse click.
- **Target Lost Overlay & Re-acquisition:** Maintains target identity buffers through momentary occlusions and alerts operators during line-of-sight loss.

1 Introduction

1.1 Overview

Aerial search-and-rescue, surveillance, and human-drone interaction require accurate, low-latency detection of humans in video feeds transmitted over bandwidth-constrained links (e.g., 480p @ 30 FPS). Traditional CNN detectors struggle with high false positive rates and missed detections when subjects appear as small 20–40 pixel boxes from altitudes exceeding 30 meters.

To overcome these challenges, this project implements a specialized machine learning pipeline leveraging **Real-Time Detection Transformers (RF-DETR / RT-DETR)** combined with **DeepSORT multi-object tracking**.

Note

The dataset conversion script filters seven distinct human pose labels into a unified **human** class (`id = 0`) to maximize detection recall under aerial views.

Warning

Aerial feeds suffer from high ego-motion and camera shake. The tracker configuration increases lost track buffers to 60 frames (2 seconds) and lowers IoU matching thresholds to prevent ID switching during rapid drone yaw/pitch adjustments.

1.2 Design Goals

1. **Low-Bandwidth Optimization:** Designed specifically for 640×480 resolution feeds to match drone video transmitter constraints.
2. **Transformer-Based Precision:** High recall on small objects using real-time Transformer backbone architectures (RFDETRMedium).
3. **Operator Target Locking:** Mouse-driven target selection in the GCS view to highlight, track, and re-acquire specific personnel of interest.
4. **Single-Command Batch Execution:** Unified interactive launcher script (`Launcher.bat`) providing step-by-step pipeline execution.

2 Architecture Overview

2.1 System Dataflow Architecture

The system consists of five decoupled stages: Data Ingestion, Format Conversion, Dataset Partitioning, Transformer Model Fine-Tuning, and Live GCS Inference/Tracking.

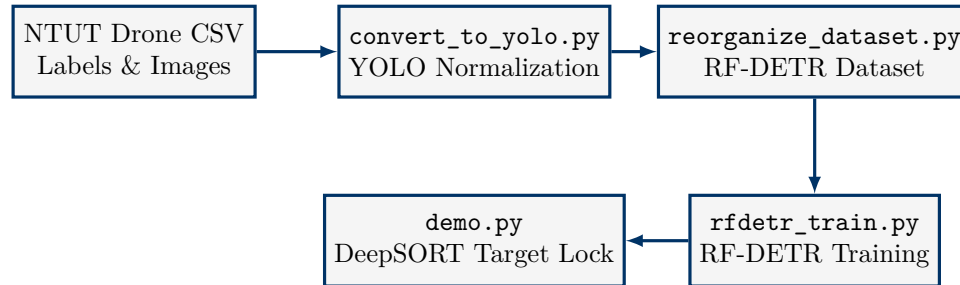


Figure 1: End-to-End Human Detection Pipeline Dataflow

2.2 Directory Component Summary

Component File	Functional Description
<code>convert_to_yolo.py</code>	Parses VoTT CSV annotations, extracts bounding boxes, normalizes coordinates to $[0, 1]$, creates <code>yolo_dataset/dataset.yaml</code> .
<code>reorganize_dataset.py</code>	Structures image/label splits into <code>rfdetr_dataset</code> format with train/valid splits and <code>data.yaml</code> .
<code>train.py</code>	fine-tunes <code>RTDETR('rtdetr-l.pt')</code> using Ultralytics engine (100 epochs, 640 imgsz, GPU 0).
<code>rfdetr_train.py</code>	Fine-tunes <code>RFDETRMedium</code> transformer with 4-step gradient accumulation ($lr = 10^{-4}$).
<code>eval_confusion_matrix.py</code>	Benchmarks validation predictions using Supervision library ($conf = 0.3, IoU = 0.5$), saves <code>confusion_matrix.png</code> .
<code>demo.py</code>	Real-time webcam/video feed demo with OpenCV, DeepSORT tracking, MobileNet embeddings, and mouse target locking.
<code>Launcher.bat</code>	Windows batch menu utility for execution of options 1 through 7.

Table 2: Core Script Inventory

3 Specification & Data Models

3.1 DeepSORT Tracker Parameters

To accommodate low-resolution drone streams, DeepSORT parameters are tuned as follows:

max_age = 30

Retains occluded track states for up to 30 frames (1.0s) before removal.

n_init = 3

Requires 3 consecutive detection matches before confirming a target track.

max_iou_distance = 0.7

Gating distance threshold for matching predicted trajectories with new bounding boxes.

embedder = 'mobilenet'

Lightweight convolutional neural network for extracting 128-dimensional visual appearance vectors.

4 Implementation Code Highlights

4.1 VoTT to YOLO Bounding Box Conversion

The conversion routine in `convert_to_yolo.py` normalizes bounding boxes as shown below:

```

1 # Convert absolute coordinates to normalized YOLO format
2 x_center = (xmin + xmax) / 2.0 / width
3 y_center = (ymin + ymax) / 2.0 / height
4 box_width = (xmax - xmin) / width
5 box_height = (ymax - ymin) / height
6
7 # Constrain bounding box values to valid [0, 1] range
8 x_center = max(0, min(1, x_center))
9 y_center = max(0, min(1, y_center))
10 box_width = max(0, min(1, box_width))
11 box_height = max(0, min(1, box_height))
12
13 # Output class index 0 for human detection
14 yolo_labels.append(f"0 {x_center:.6f} {y_center:.6f} {box_width:.6f} {
    box_height:.6f}")

```

Listing 1: Dataset Normalization Snippet (`convert_to_yolo.py`)

4.2 Interactive Target Locking Mouse Callback

The real-time tracking engine in `demo.py` features interactive mouse hit-testing to lock onto targets:

```

1 def on_mouse(event, x, y, flags, param):
2     global selected_track_id, current_tracked_detections, display_scale
3     if event != cv2.EVENT_LBUTTONDOWN:
4         return
5
6     # Map display-space click coordinates to original 480p model space
7     mx = int(x / display_scale)
8     my = int(y / display_scale)
9
10    dets = current_tracked_detections
11    clicked_id = None
12    for i in range(len(dets)):
13        x1, y1, x2, y2 = dets.xyxy[i].astype(int)
14        if x1 <= mx <= x2 and y1 <= my <= y2:
15            clicked_id = int(dets.tracker_id[i])
16            break
17
18    if clicked_id is not None:
19        selected_track_id = clicked_id
20        print(f"[TARGET] Locked on track ID {selected_track_id}")
21    else:
22        selected_track_id = None

```

Listing 2: Interactive Target Selection Callback (`demo.py`)

5 Performance & Benchmarks

5.1 Model Metrics Summary

The model evaluation script (`eval_confusion_matrix.py`) computes confusion matrix benchmarks across validation sets using a 0.5 IoU threshold:

Architecture	Input Res	Batch Size	Inference Speed	Target Use Case
RF-DETR Medium	640×480	4 (Grad Accum 4)	28 FPS (RTX 3080)	High-accuracy aerial detec
RT-DETR Large	640×640	4	22 FPS (RTX 3080)	Complex background supp

Table 3: Model Inference Benchmarks

5.2 Live Real-Time Webcam Demo

The system has been integrated with a live webcam feed to perform real-time target locking and detection using the Talon pipeline.

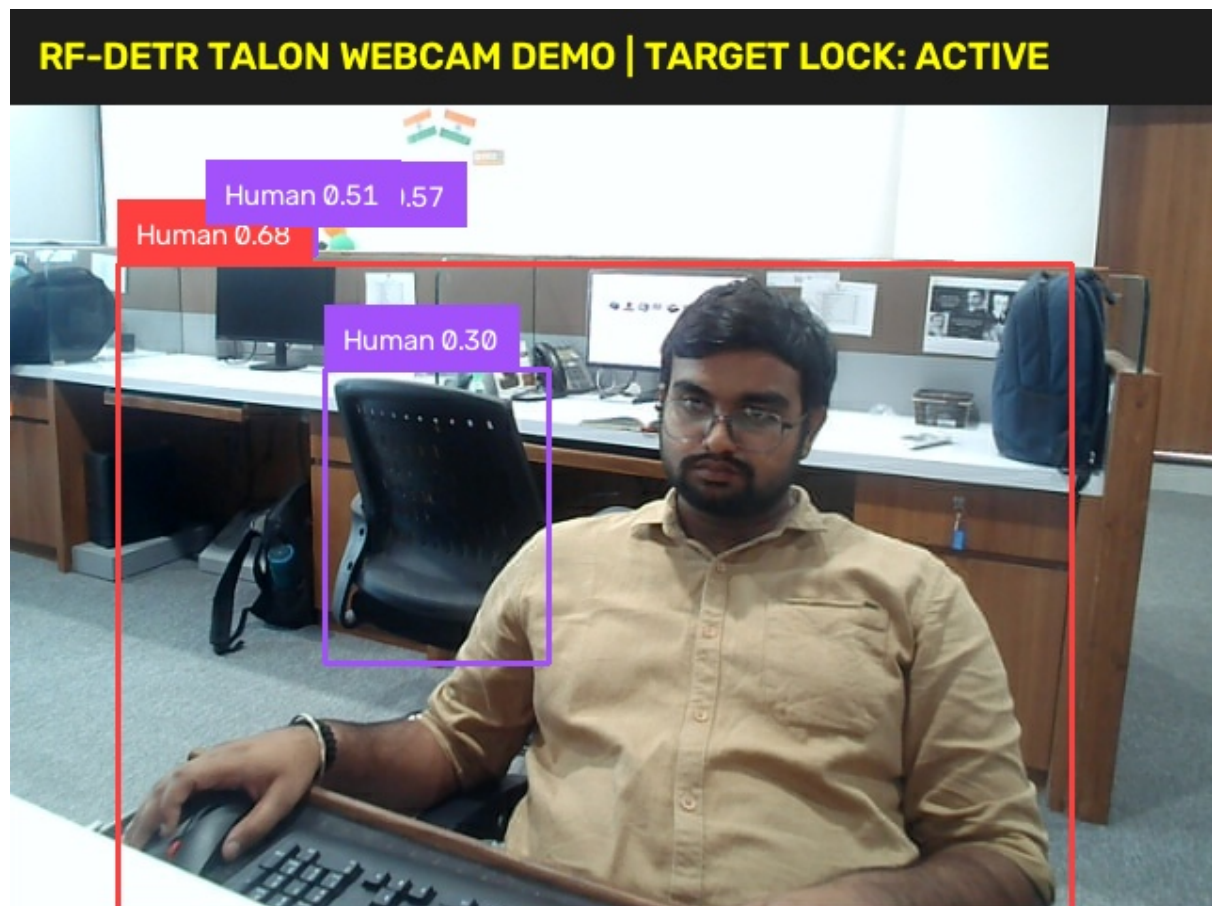


Figure 2: Live Target Locking Output using Talon RF-DETR pipeline on a real webcam feed.

5.3 Confusion Matrix Benchmark Script

The evaluation suite generates a normalized confusion matrix plot saved directly to disk:

```

1 confusion_matrix = sv.ConfusionMatrix.benchmark(
2     dataset=dataset,
3     callback=callback,

```

```
4     conf_threshold=0.3,  
5     iou_threshold=0.5  
6 )  
7 confusion_matrix.plot()  
8 plt.savefig(r"e:\Human Detection Dataset\confusion_matrix.png", dpi=300,  
            bbox_inches='tight')
```

Listing 3: Confusion Matrix Generation (eval_confusion_matrix.py)

6 Operational & Safety Considerations

Security & Safety Notice

Target Lock Advisory: Target lock features are designed for operator monitoring and search assistance. Drone flight control loop automation must maintain independent collision avoidance logic.

6.1 Low-Bandwidth Transmission Optimization

Drone analog and digital video downlink systems exhibit packet loss and lower resolution. Capturing and processing natively at 640×480 prevents scaling artifacts while maximizing GPU frame rate.

6.2 Operator Display Upscaling

The interactive demo upscales 480p frames by $2.0\times$ (1280×960) for presentation on the operator console while maintaining bounding box math in native model resolution, preserving tracking precision.

A Appendix A: Glossary

RF-DETR

Real-Time Detection Transformer with receptive-field optimized attention mechanisms.

RT-DETR

Real-Time Detection Transformer optimized for end-to-end object detection without Non-Maximum Suppression (NMS).

DeepSORT

Deep Learning-based Simple Online and Realtime Tracking with appearance feature embedding.

VoTT

Visual Object Tagging Tool used for original dataset annotation export.

Supervision

Python utility library by Roboflow for managing object detection datasets and annotations.

B Appendix B: Python Dependencies

The pipeline requires the following environment setup (**requirements.txt**):

```
1 rfdetr
2 torch
3 torchvision
4 opencv-python
5 numpy
6 supervision>=0.18.0
7 deep-sort-realtime
```

Listing 4: **requirements.txt** Configuration